Лабораторна робота №4

Безперервна інтеграція та автоматизація розгортання (CI/CD)

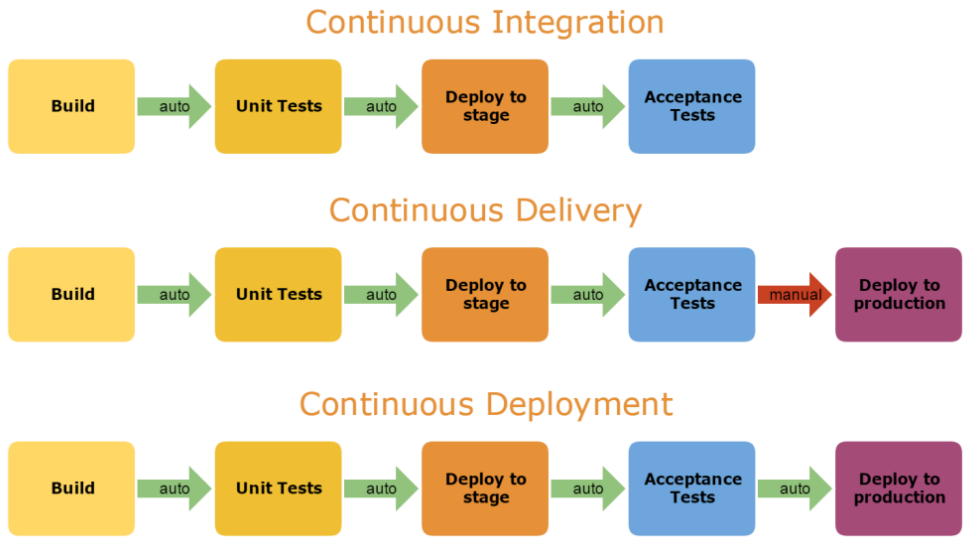
Мета роботи: Опанувати принципи та інструменти безперервної інтеграції (CI) та безперервної доставки (CD). Навчитися описувати життєвий цикл проєкту у вигляді коду (Pipeline as Code), автоматизувати перевірку якості та забезпечити автоматичне розгортання MVP у хмарному середовищі.

1. Автоматизація життєвого циклу розгортання та інструменти CI/CD

Автоматизація збірки та розгортання є критично важливою частиною сучасного життєвого циклу розробки програмного забезпечення, що дозволяє перетворити сирцевий код у високопродуктивний та стабільний продукт. Використання спеціалізованих інструментів мінімізує рутинні операції, забезпечує надійну перевірку якості та готує артефакти до розгортання в різних середовищах.

Концепція CI/CD

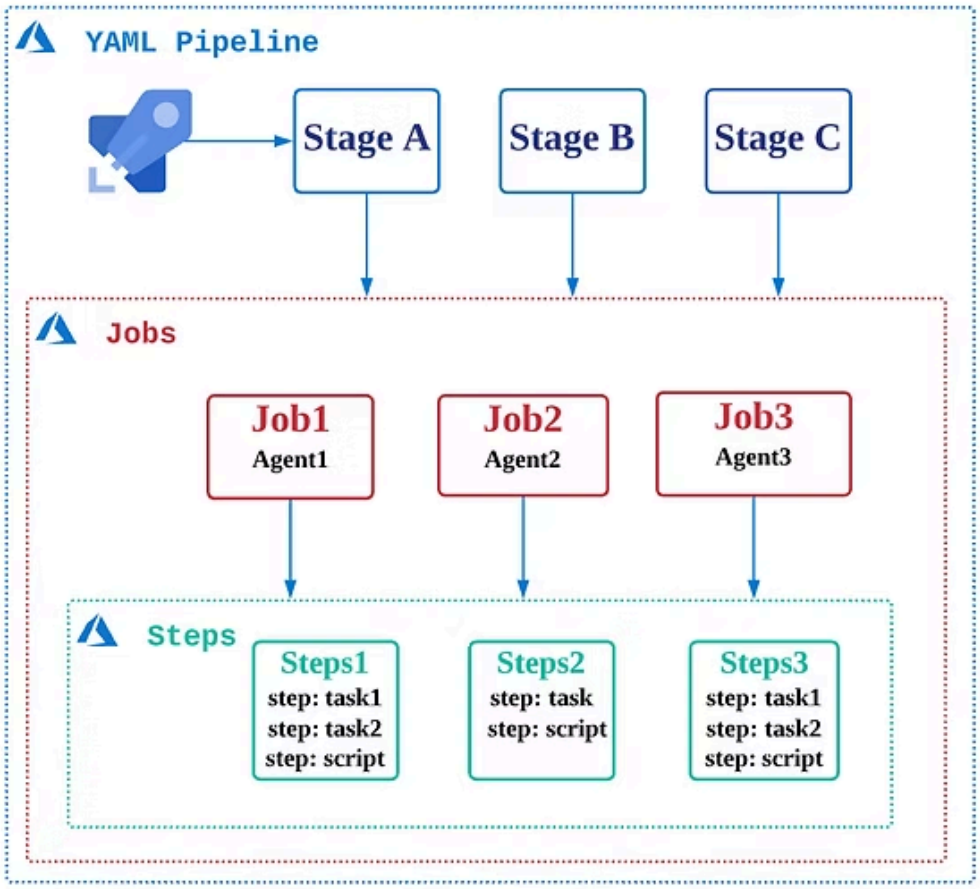
Це комбінація практик, що автоматизують шлях коду від репозиторію до сервера.



- **Continuous Integration (CI):** Практика автоматичної перевірки коду (лінтування, тести) при кожному оновленні репозиторію. Мета — виявити помилки інтеграції якомога раніше.
- **Continuous Delivery (Безперервна доставка):** Розширення CI, де після успішних тестів автоматично створюється готовий до випуску артефакт. Розгортання на "прод" відбувається за командою людини.
- **Continuous Deployment (Безперервне розгортання):** Повністю автоматизований цикл, де кожна зміна, що пройшла всі перевірки, негайно потрапляє до кінцевого користувача без ручного втручання.

YAML-конфігурація

Спеціальний формат файлів (часто .yml або .yaml), що використовується для опису алгоритмів автоматизації (Pipelines) як коду.



- **Stages (Етапи):** Найвищий рівень логічного групування в пайплайні. Етапи зазвичай розділяють великі фази життєвого циклу, як-от «Збірка» (Build), «Тестування» (Test) та «Розгортання» (Deploy). Наступний етап часто не починається, поки попередній не завершиться успішно.
- **Jobs (Завдання):** Групи кроків, які виконуються всередині одного етапу. Важливо, що завдання всередині етапу можуть виконуватися паралельно на різних віртуальних машинах

(Runners), що значно пришвидшує загальний час виконання.

- **Steps (Кроки):** Найменші одиниці роботи всередині завдання. Це послідовні дії, як-от запуск конкретного скрипту, встановлення бібліотеки або виконання команди npm run build.
- **Triggers (Тригери):** Події, що запускають процес (наприклад, push у гілку release або створення Pull Request).

Runners та Environments

Технічне середовище, де відбувається виконання інструкцій автоматизації.

- **Runners (Раннери):** Віртуальні машини або контейнери (наприклад, у хмарі GitHub чи GitLab), які надають обчислювальні потужності для виконання вашого YAML-файлу.
- **Secrets & Variables:** Механізм безпечного зберігання чутливих даних (API-ключі, паролі до БД). Секрети ніколи не зберігаються у відкритому коді, а прокидаються в Runner як змінні оточення під час збірки.

Статус-бейджі (Status Badges)

Візуальні індикатори (невеликі зображення-посилання), що розміщуються в README.md. Вони в реальному часі відображають успішність останньої збірки або відсоток покриття коду тестами, забезпечуючи прозорість стану проєкту для всієї команди.

Service	Master	Develop
CI Status	PASSED	PASSED
Test coverage	codecov 24%	codecov 46%
Test report	Test Report master	Test Report develop
Documentation	Doxygen master	Doxygen develop

Deployment Providers

Сервіси, що спеціалізуються на автоматичному хостингу та розгортанні сучасних вебзастосунків.

- **Vercel / Netlify:** Орієнтовані на фронтенд-застосунки. Забезпечують атомарні деплої, автоматичне створення прев'ю для кожної гілки та миттєвий відкат до попередніх версій.
- **GitHub Pages:** Просте рішення для хостингу статичних сайтів безпосередньо з репозиторію GitHub.

2. Індивідуальне завдання

Крок 1: Створення Workflow

У корені проєкту створити папку `.github/workflows/` .

Створити файл `main.yml` (або `ci-cd.yml`).

Налаштувати тригер: запуск при кожному `push` у гілку `main` або `develop` та при кожному `pull_request` .

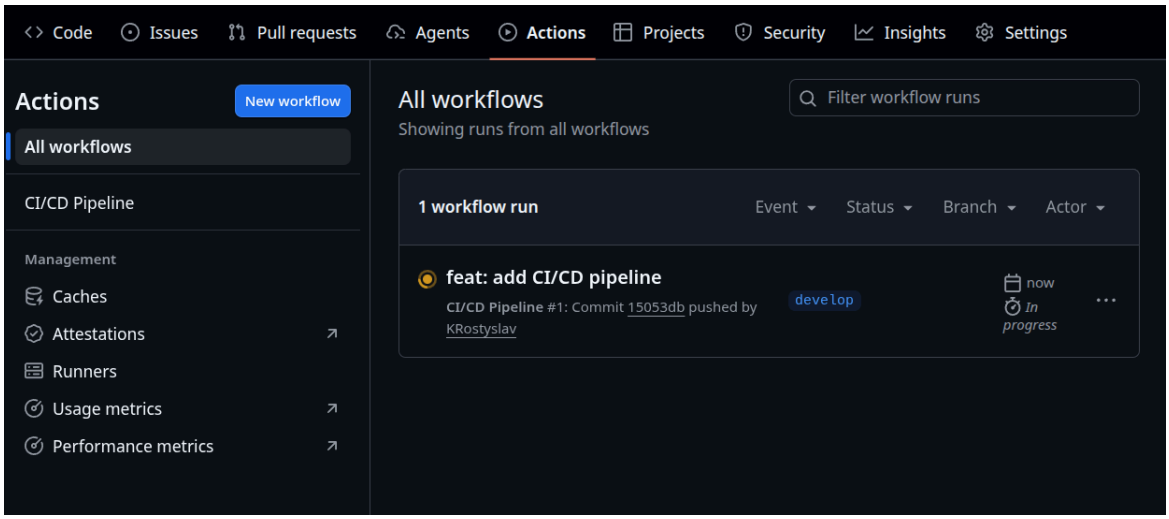
Пояснення:
Папка `.github/workflows/` — це спеціальна директорія, де GitHub шукає інструкції для автоматизації. Файл у форматі YAML визначає конфігурацію вашого CI/CD процесу.
Секція `on` (тригери) відповідає за те, *коли* запускати перевірку:

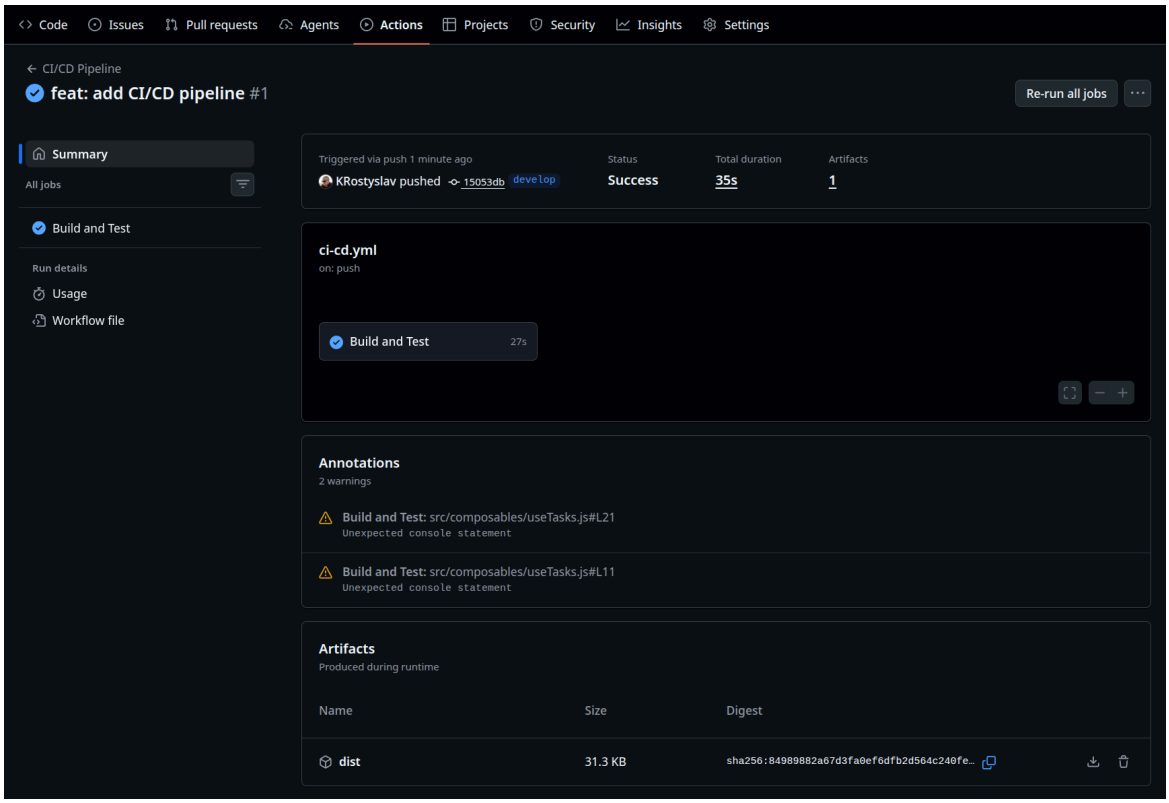
- `push` у `main` / `develop` : гарантує, що основні гілки завжди містять перевірений код.
- `pull_request` : дозволяє виявити помилки в коді ще на етапі рецензування (Code Review), до злиття змін.

ci-cd.yml

.github > workflows > ci-cd.yml

```yaml
name: CI/CD Pipeline

# Тригери для запуску пайплайну
on:
  # Запуск при push у гілки main та develop
  push:
    branches:
      - main
      - develop
  # Запуск при створенні Pull Request
  pull_request:
    branches:
      - main
      - develop

jobs:
  # Job для збірки та тестування
  build-and-test:
    name: Build and Test
    runs-on: ubuntu-latest

    steps:
      # Крок 1: Отримання коду з репозиторію
      - name: Checkout repository
        uses: actions/checkout@v4

      # Крок 2: Налаштування Node.js
      - name: Setup Node.js
        uses: actions/setup-node@v4
```





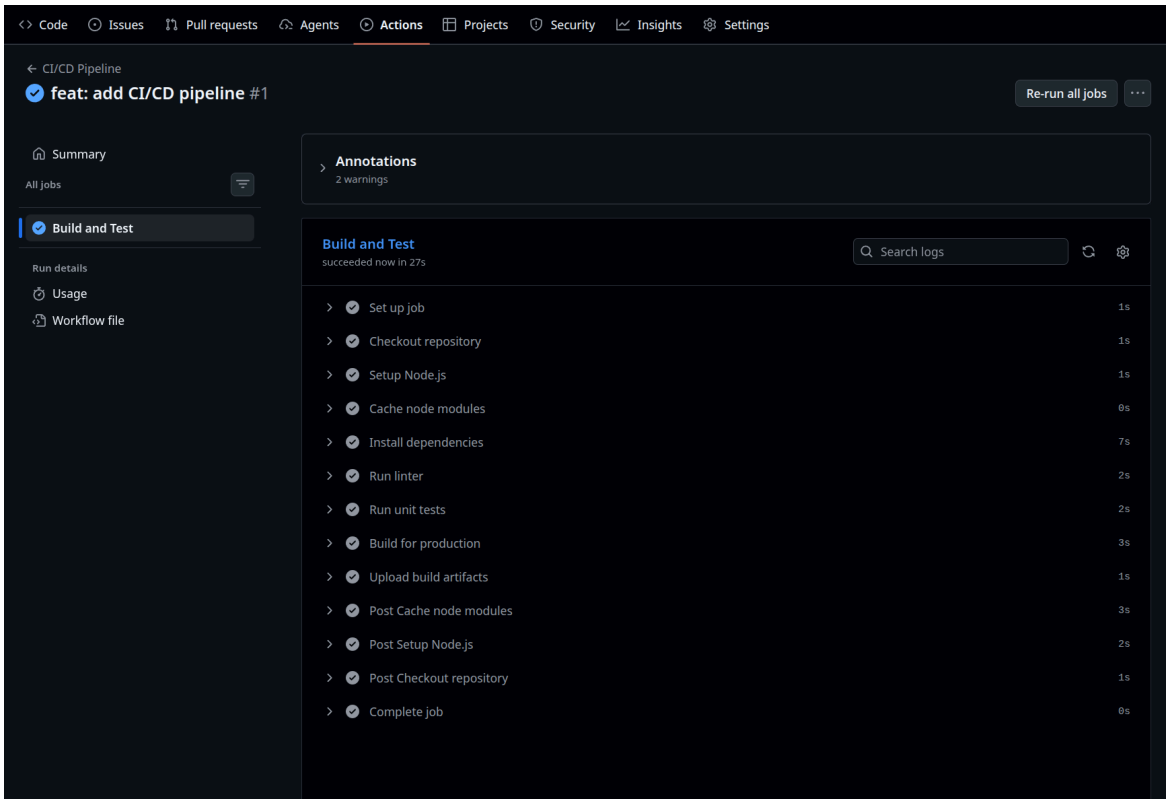
Крок 2: Налаштування етапу перевірки (CI)

На цьому етапі ми описуємо завдання (`job`), яке буде виконуватися автоматично. Назвемо його `build-and-test` .

Це завдання запустить віртуальну машину (runner), встановить необхідні інструменти та виконає перевірку коду.

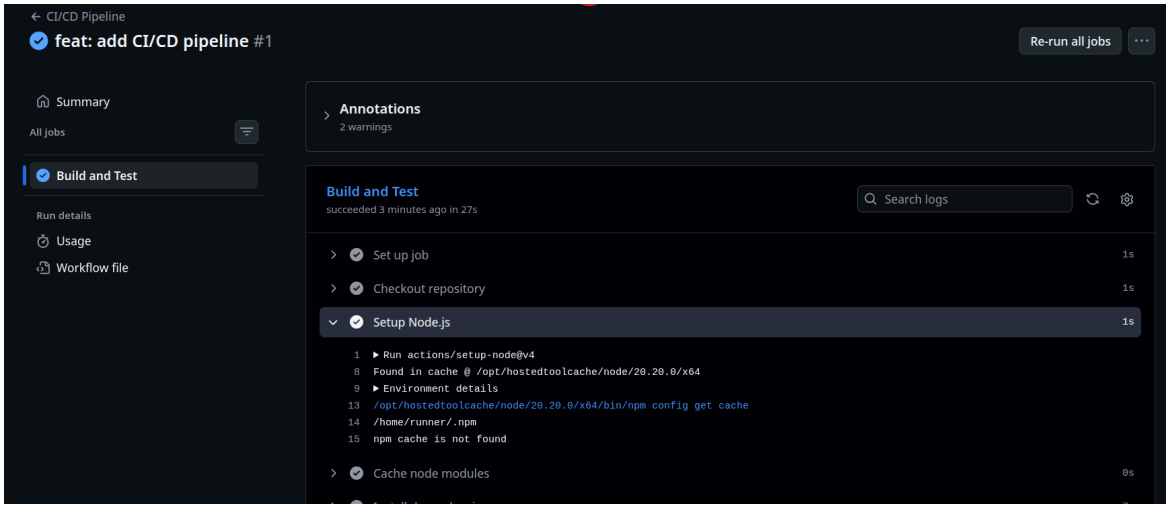
1. Визначення середовища виконання:

Ми будемо використовувати `ubuntu-latest` — це стандартний образ Linux для GitHub Actions, який містить необхідні утиліти.



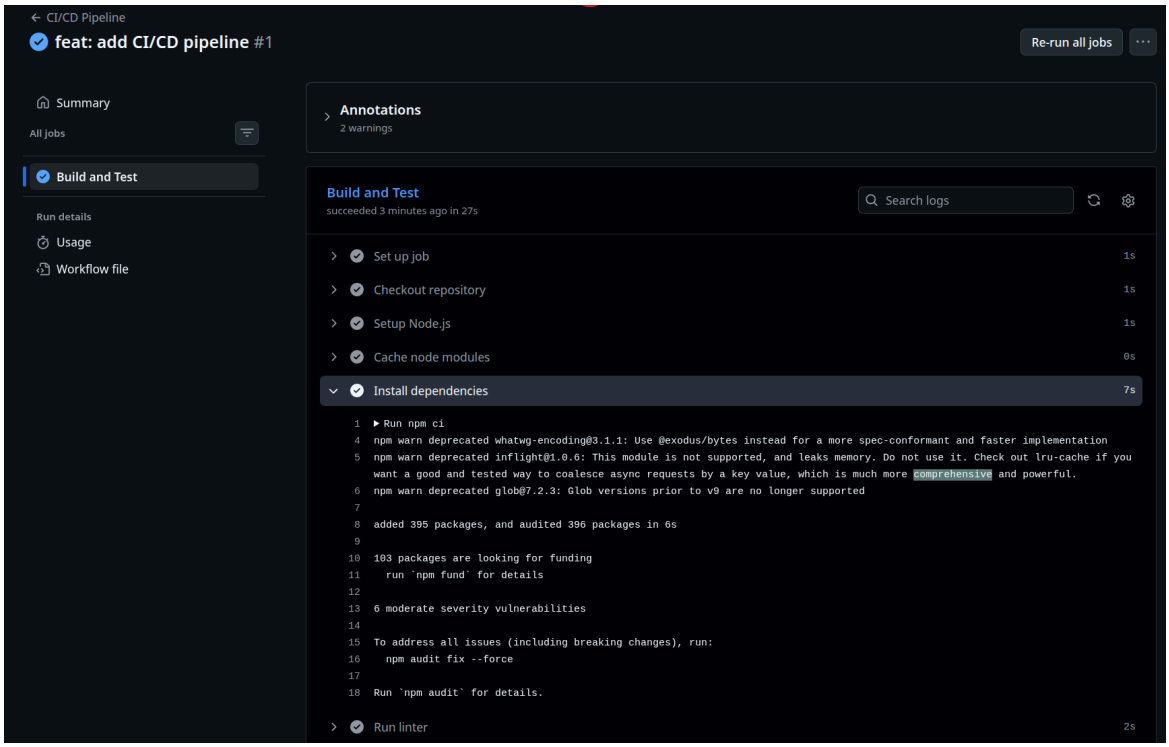
2. Налаштування оточення (Node.js):

Використовуємо екшен `actions/setup-node`, щоб встановити потрібну версію Node.js (наприклад, 18 або 20). Це гарантує, що тести будуть запускатися в стабільному, прогнозованому середовищі, незалежно від налаштувань локального комп'ютера розробника.



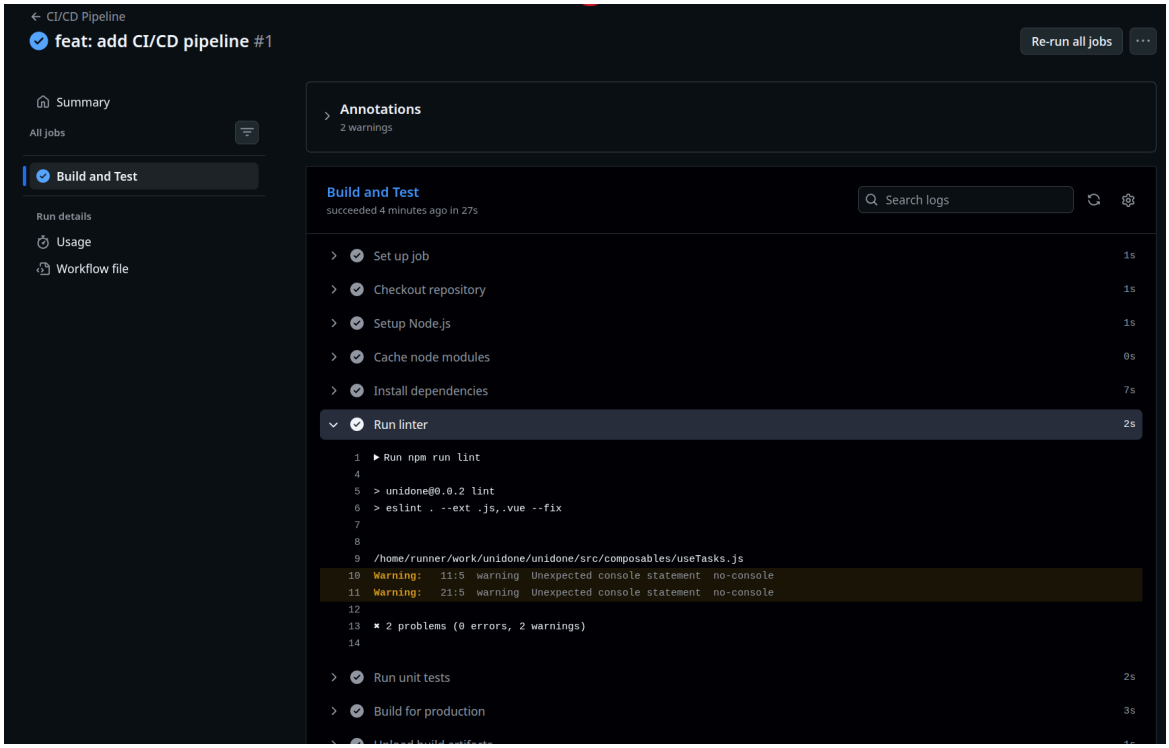
3. Встановлення залежностей:

Команда `npm ci` (clean install) або `npm install` завантажує бібліотеки з `package-lock.json`. Це критично для відтворюваності білда.



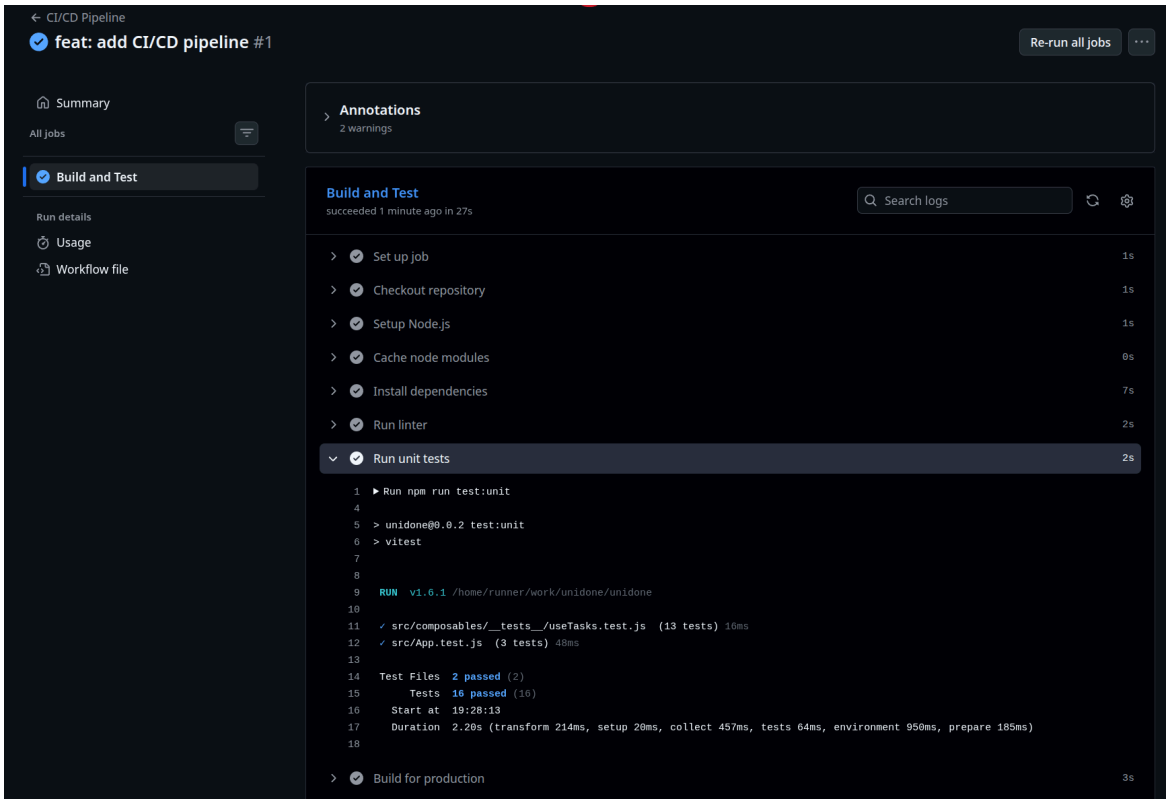
4. Перевірка якості коду (Linting):

Запускаємо `npm run lint`. Лінтер перевіряє синтаксис та стиль коду. Якщо є помилки форматування — пайплайн автоматично зупиниться і не дозволить злити неякісний код.



5. Запуск тестів:

Виконуємо `npm run test:unit`. Якщо хоча б один тест впаде, весь процес буде позначено як "Failed" (червоний хрестик). Реальні проєкти не допускають деплою з поламаними тестами.

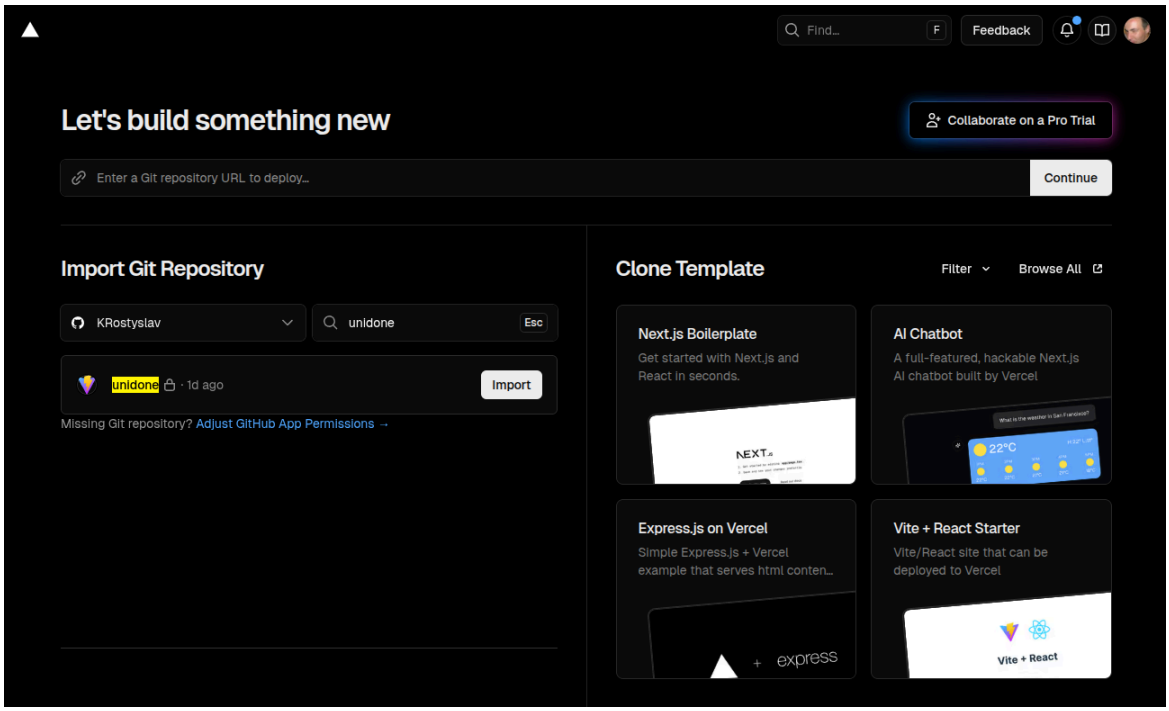


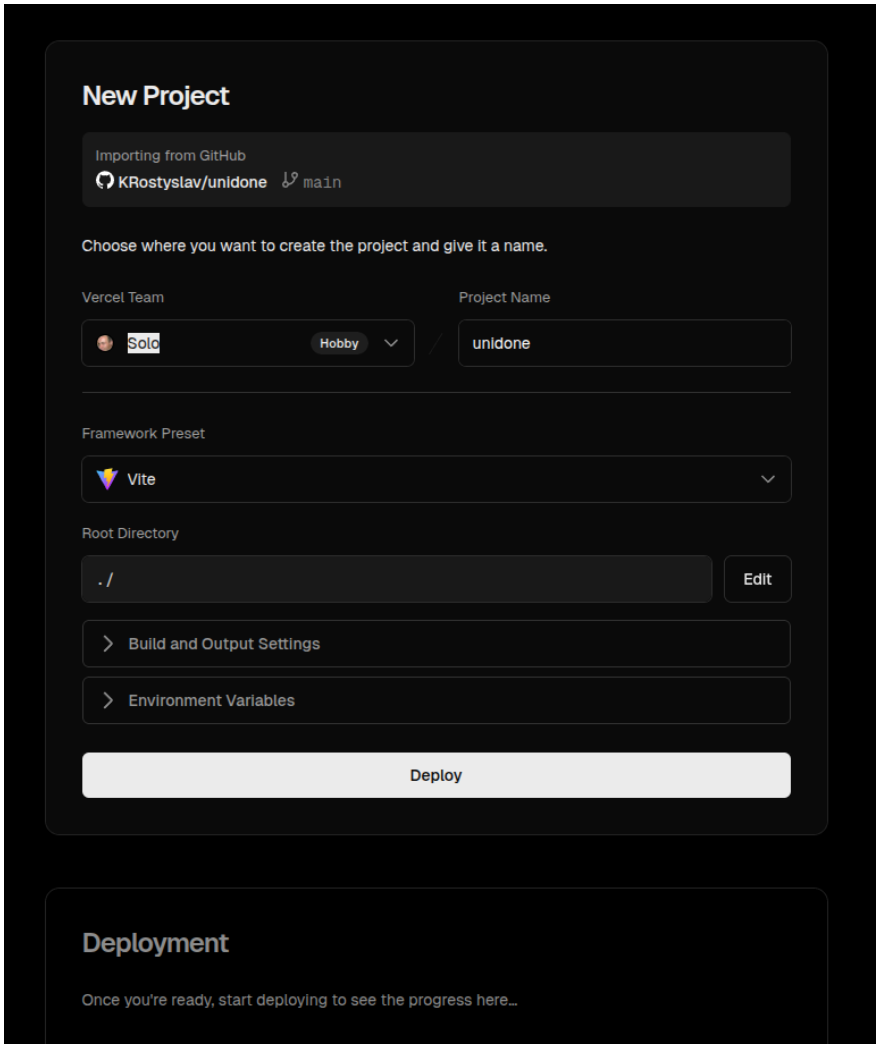
> **Завдання:** Перевірити, чи стає Pipeline "червоним", якщо навмисно зламати тест або порушити правила лінтера.

Крок 3: Налаштування автоматичного деплою (CD)

1. Інтеграція з хостингом (Vercel):

Vercel дозволяє налаштувати Continuous Deployment (безперервне розгортання) вашого фронтенд-застосунку.
Необхідно створити аккаунт на Vercel та імпортувати репозиторій проєкту.

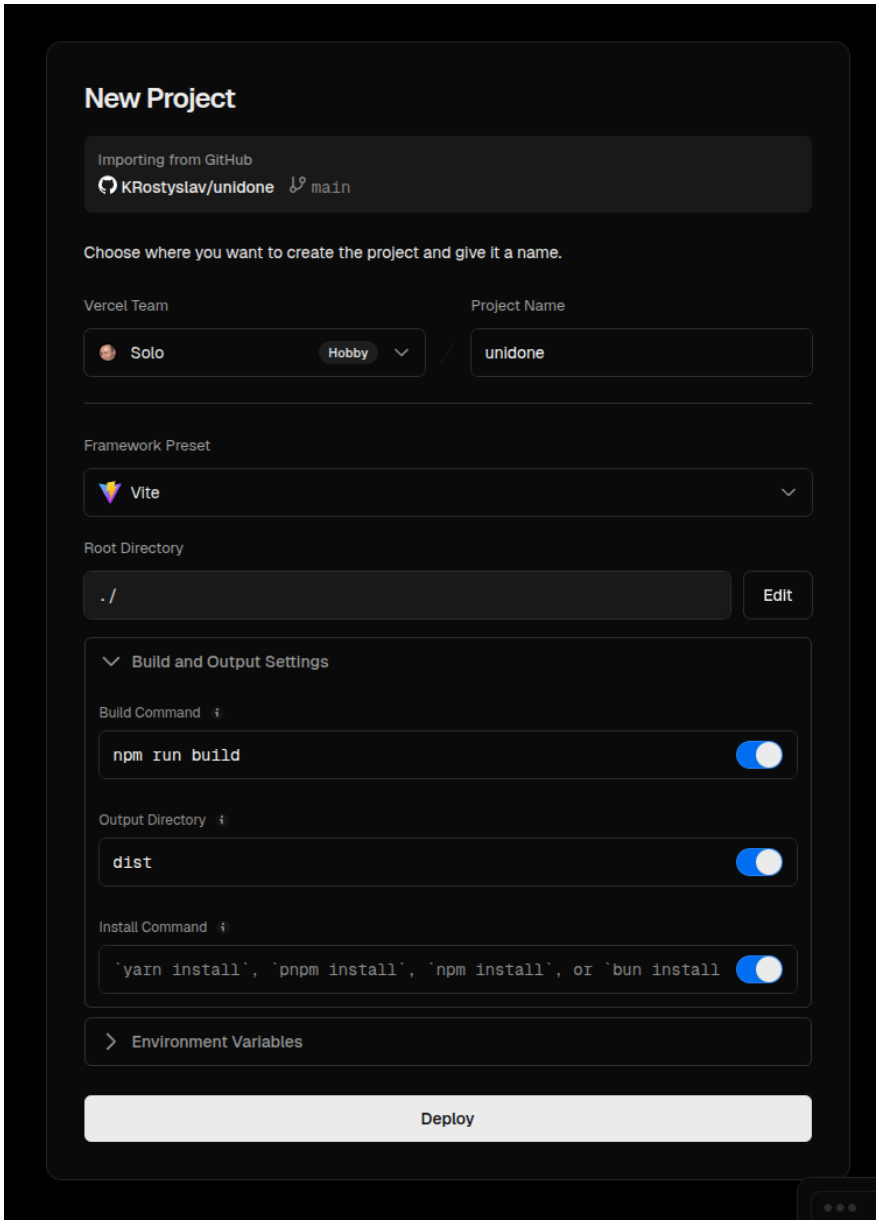




Важливо: У налаштуваннях "Build & Development Settings" переконайтеся, що Framework Preset встановлено як **Vite**, а команда збірки (Build Command) — `npm run build` . Це навчить сервер

хостингу збирати проєкт автоматично при отриманні нового коду.

Vercel автоматично виявить зміни в гілці `main` і запустить процес оновлення сайту.



Environment Variables

Deploying...

Deployment

Deployment started 8s ago...

Build Logs `Installing dependencies ...` 6s

Deployment Summary

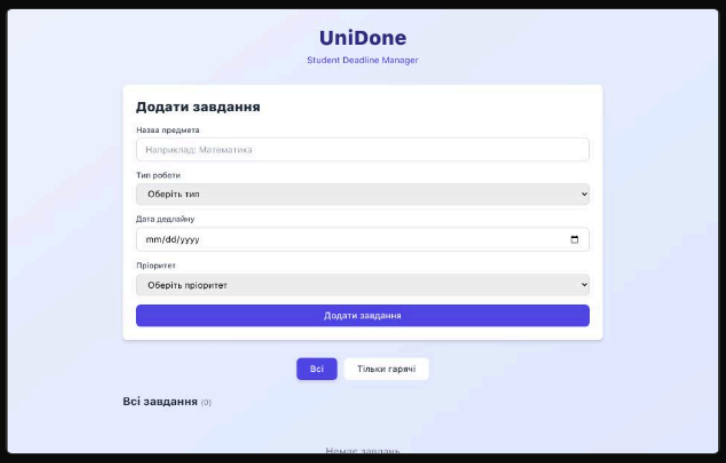
Assigning Custom Domains

feat: initial commit - MVP UniDone project setup - 3d19b0

Cancel Deployment

Congratulations!

You just deployed a new project to Solo.



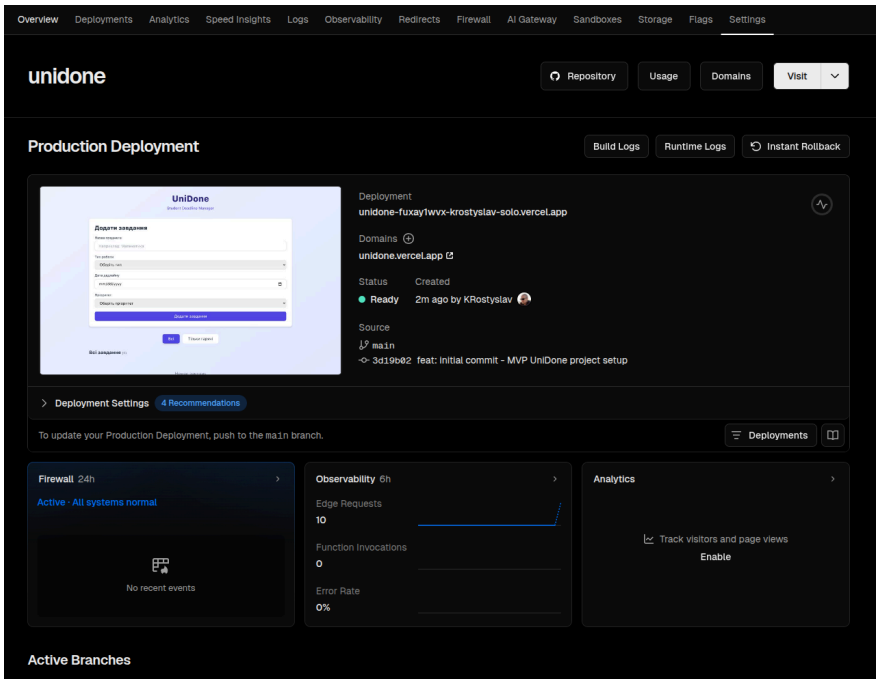
Next Steps

Instant Previews
Push a new branch to preview changes instantly

Add Domain
Add a custom domain to your project

Enable Speed Insights
Track how users experience your site over time

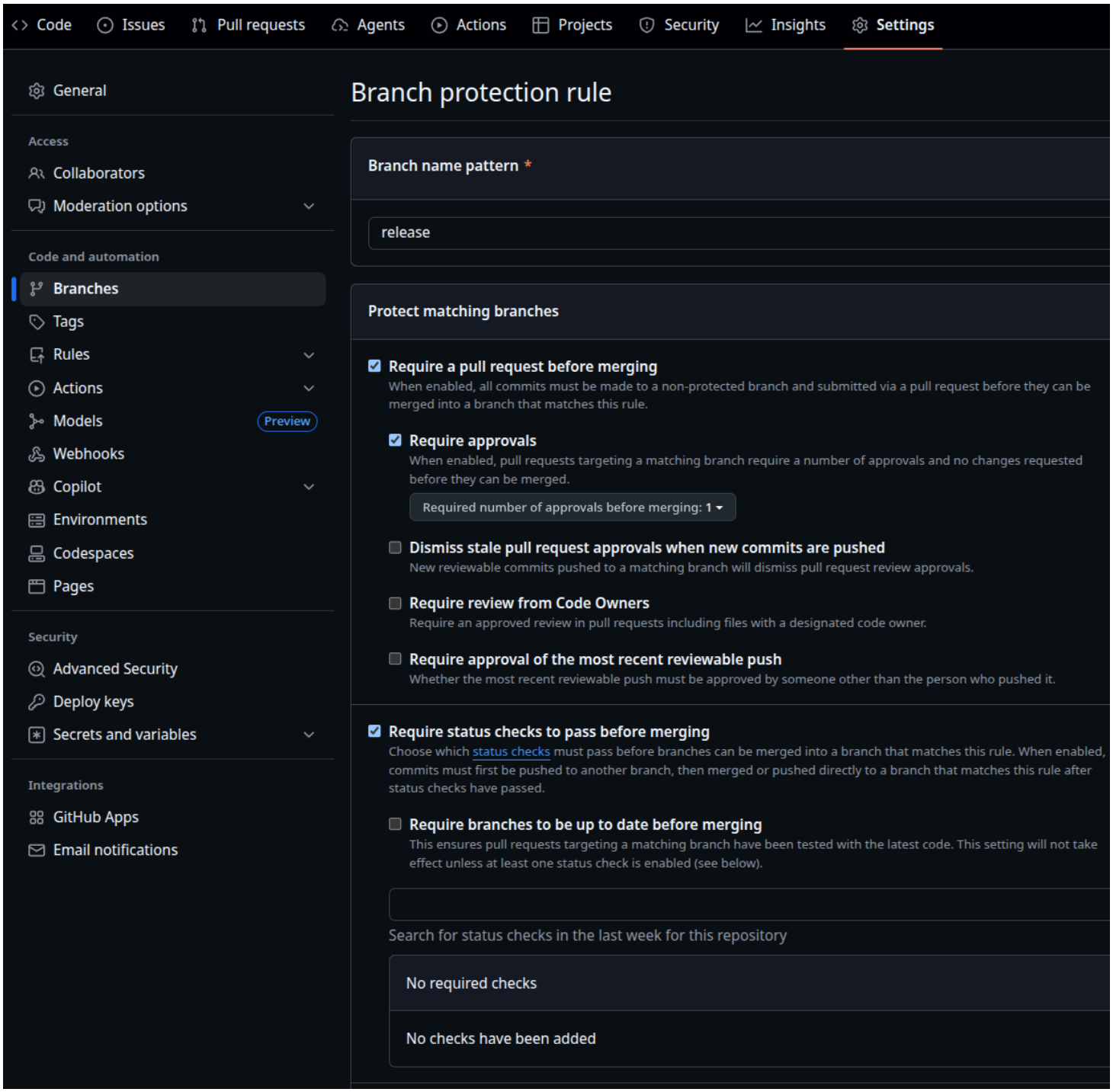
Continue to Dashboard



2. Залежність від перевірок (Quality Gate):

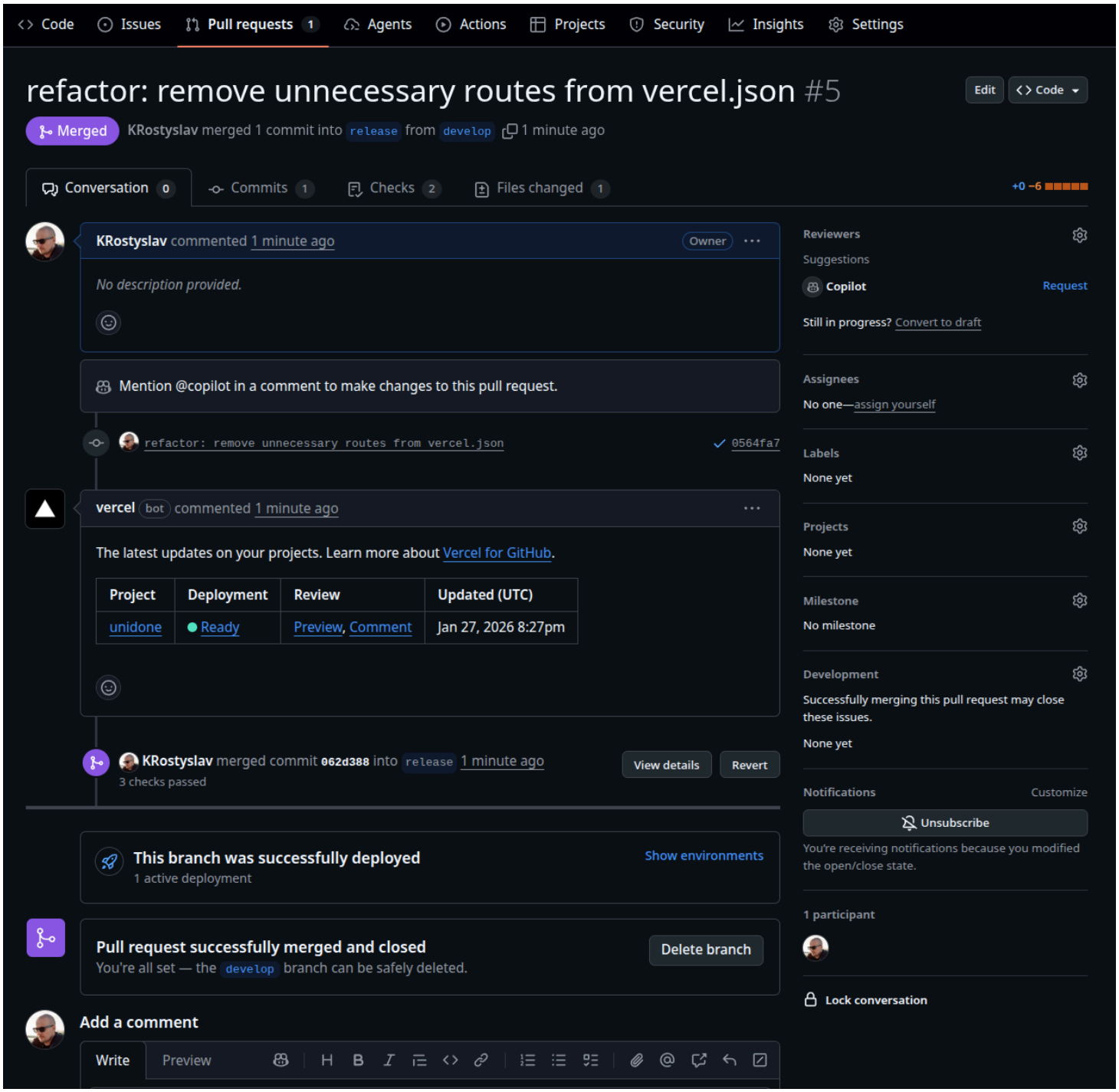
Автоматичний деплой — це зручно, але ризиковано, якщо деплоїти зламаний код. Ми створимо "ворота якості" (Quality Gate).
Щоб деплой відбувався лише при успішних тестах, у GitHub Actions переконаємося, що всі перевірки проходять успішно.

У налаштуваннях репозиторію на GitHub (**Settings > Branches**) створити Branch Protection Rule для гілки `main` .



Увімкнути опцію **"Require status checks to pass before merging"** та обрати ваш Job (`build-and-test`).

Що це дає? Це гарантує, що неякісний код (який не пройшов тести або лінтер) фізично неможливо буде злити (merge) у гілку релізу, і він не запустить автоматичну збірку на Vercel.



Крок 4: Робота з секретами (за потреби)

У реальних проєктах часто потрібно використовувати конфіденційні дані (API-ключі, токени доступу до хмарних сервісів, паролі баз даних), які **категорично заборонено** зберігати у відкритому коді.

1. Додавання секрету:
Перейдіть у налаштування репозиторію: **Settings > Secrets and variables > Actions**.
Натисніть **New repository secret**.

- **Name:** Назва змінної (наприклад, `VERCEL_TOKEN` або `DATABASE_PASSWORD`).
- **Secret:** Сама секретна строка.

2. Використання у Workflow:
GitHub автоматично шифрує ці дані. У вашому YAML-файлі ви можете звертатися до них через спеціальний контекст:
`${{ secrets.NAZVA_VASHOGO_SEKRETU }}`.
GitHub Actions підставить реальне значення під час виконання, але в логах воно буде приховане зірочками (`***`).

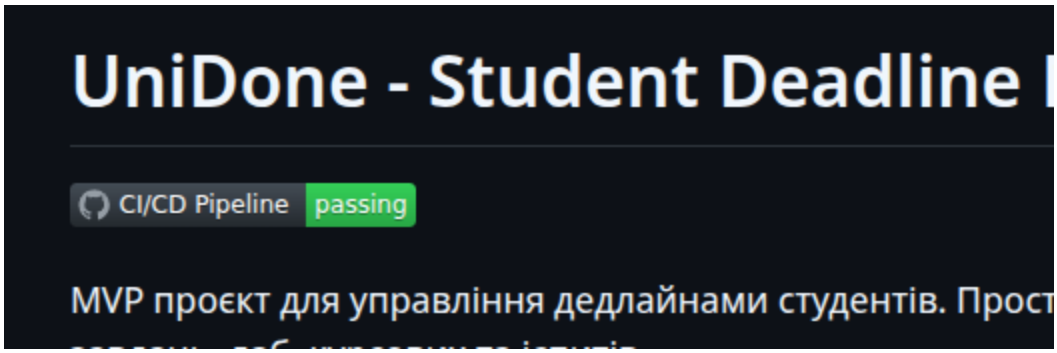
Крок 5: Статус-бейдж та фіналізація

Статус-бейдж — це динамічна картинка, яка показує поточний стан вашого проєкту (успішно зібрався / впав). Це стандарт де-факто для Open Source проєктів.

1. Отримання бейджа:

1. Перейдіть на вкладку **Actions**.
2. В лівій панелі оберіть ваш workflow (наприклад, "CI/CD Pipeline").
3. Натисніть кнопку **... (три крапки)** справа зверху -> **Create status badge**.
4. Скопіюйте **Markdown** код.

2. Додавання у README:
Вставте скопійований код у самий верх вашого файлу `README.md`.



3. Посилання на продакшн:
Обов'язково додайте посилання на "живу" версію вашого застосунку (Production URL, який надав Vercel) у опис репозиторію (розділ "About" справа на головній сторінці GitHub) та у `README.md`.
Це дозволить викладачу та користувачам одразу побачити результат вашої роботи.

3. Вимоги до звіту

1. Скриншот вкладки Actions у GitHub, де видно успішну історію запусків (всі галочки зелені).
2. Скриншот самого YAML-файлу з поясненням кожного кроку.
3. Посилання на репозиторій з бейджем у README.md.
4. Головний результат: Посилання на робочий веб-сайт проєкту UniDone у мережі Інтернет.

4. Контрольні запитання

1. Яка різниця між on: push та on: pull_request у GitHub Actions?
2. Чому важливо виконувати npm run build саме в CI/CD пайплайні, а не просто завантажувати папку dist/ у репозиторій?
3. Що таке "секрети" репозиторію і чому їх не можна додавати у Git?
4. Як автоматизація деплою впливає на показник Time to Market (час виходу продукту на ринок)?